

web3d-lab • 3DGS 在线重建查看器 + 个人主页

郑州大学 双足实验室（智能机器人创新训练基地）招新第一轮考核作品 · 方向：3D（三维视觉 / 三维重建）
作者：肖宇航（软件工程 2026 级） 线上：www.shawnlab.cn 源码：github.com/Shawn100861224/web3d-lab
联系：18711505157@163.com

10 个
线上可浏览场景（4 官方 + 6 自训）

60/60
实拍照片 COLMAP 注册率（100%）

30,000 步
云端 GPU 训练步数

66,392
线上场景实际高斯点数

15.41 dB
实拍场景留出视角 PSNR

39 项
后端自动化测试（全绿）

这是什么
一个前后端一体、已经真的在公网上跑的作品站。访客打开链接，能在浏览器里实时旋转我用手机拍的真实物体训练导出的三维高斯泼溅（3DGS）场景，并在旁边看到该场景的真实训练指标（PSNR / SSIM / 高斯点数 / 训练耗时 / 显存占用）——这些数字由训练脚本回写进数据库，不是写死的。访问统计与留言板是活的：数据写在后端数据库里，刷新页面看到的是真实累计值。考核里多数人交静态个人主页；能把「自己拍、自己训出来的重建场景」放进网页实时渲染、并且后端真上线的，几乎没有，这是这个作品的全部立足点。

首页	精选场景 + 访问统计（实时，含 14 天曲线）	场景库	10 个场景卡片，缩略图 / 点数 / 指标，数据来自接口
查看器	Three.js + Spark 加载 .splat / .spz，轨道漫游 + 自动巡航 + 实时指标面板	方法对比	3DGS 家族 9 类方法 × 8 个维度（基元/表面/规模/速度/压缩/开源/"为什么和我有关"）
关于我	方向理解、真实做过的事、踩过的坑、116 个仓库的项目雷达（真实抓取数据）、用 AI 的过程、下一步	留言板	在线可写（限流 + 链接拦截 + 注入防护）；12 条留言板测试覆盖写入与反垃圾链路

前端	Vite 8 + React 19 + TypeScript	3D 渲染	Three.js + @sparkjsdev/spark
后端	FastAPI + SQLAlchemy, EdgeOne Cloud Functions (Python)，线上 Neon Postgres、本地 SQLite	训练	COLMAP (SfM) + gsplat；本机 RTX 5060 Laptop 8GB，重活租云端 GPU (AutoDL vGPU-32GB)
测试	pytest（后端 39 项）+ 截图像素级渲染核验脚本	部署	自有域名 www.shawnlab.cn（腾讯云 EdgeOne·免备案）+ GitHub Pages 镜像，CLI 自动部署

明确不用 Electron：交付形态是「一个链接」——安装包手机打不开、对 3D 零增益。

部署	edgeone makers deploy 一条命令整包上线（静态站 + 云函数同域，免 CORS）；GitHub Actions 自动同步镜像
质量门	CI 两个 job 每次提交自动跑（前端构建 + 后端 39 项测试，约 27 秒）；verify-cloud.sh 七项自检可重复执行
可观测	站点健康检查每天 09:30 自动跑，异常才响；访问统计按天分桶、30 秒去重、独立访客去重
分享	每个场景深链生成自己的分享卡片（微信/QQ 里点开显示对应场景的标题与图，不是一个通用封面）
性能	查看器按需加载：首页 JS 从 3.4 MB 降到 316 kB (gzip 104 kB)，缩小 91%，首页不再下载那 1 MB 的 3D 引擎
可访问性	按 Vercel Web Interface Guidelines 逐条体检后修复：键盘可见焦点环、跳到主内容、表单提交结果的读屏播报
留存	训练产物（.ply / .splat / 训练日志/指标）除本地外，另存一份到 GitHub Release（公开可下载，实测 200）
合规	代码 MIT 许可；自训资产与官方示例的归属在 README 里写清（不冒用他人资产）

- 我具体做了什么（AI 做了什么、我否掉了什么）
- 先定结构再让 AI 写：先想清页面要回答评审的四个问题，把每节内容写成中文要点，再让 AI 实现版式——反过来会得到漂亮但没有信息的页面。
 - 不让 AI 编数据：星标是脚本抓的；3DGS 的点数、PSNR、帧率是跑出来的；上游示例没有指标就如实写“未测”，不套用论文数字。
 - AI 的方案被我退回两次：项目雷达先用爬虫抓 GitHub → 我改成官方 API；接口限额没处理 → 改成已授权令牌 + 结果落文件。

- 发现筛选逻辑不严，自己补规则：抓回 138 个仓库里混着人脸识别、扫地机器人等不相关项目 → 我加「相关性关键词 + 人工否决清单」，重抓到 116 个。
 - 不接受“跑起来了就算完成”：要求每步留下证据——注册率不足就报错、空画布不许算通过（用像素统计判定）。
 - 主动放弃并记录：全分辨率 30000 步实测每步 3.6 秒（约 30 小时）→ 停掉，把数字记进文档，而不是硬跑。

- 怎么验证的（都是真实运行的输出，不是“应该没问题”）
- 训练链路端到端跑通：手机照片 → COLMAP 稀疏重建 → 云端 GPU 上 gsplat 训练 30000 步（渲染分辨率 2016×1512）→ 导出上网版资产 → 网页实时渲染 → 指标回写数据库。
 - 三组对照实验，不是无限调参：同一批数据在云端跑三组（半分辨率全量稠密化 64.8 万高斯 / 全分辨率保守稠密化 1.5 万高斯 / 全分辨率中等稠密化 6.7 万高斯），留出视角 PSNR 分别为 14.46 / 14.52 / 14.44 —— 显存从 1.5 GB 换到 31 GB，画质几乎没动，据此把瓶颈定位到拍摄而非算力，避免了继续烧钱。
 - 两次有针对性的改进，各值多少也如实记了：① 后处理清掉浮动点与大团子（视觉更干净，细节不变多）；② 剔除 6 张最糊的照片重训（留出视角 14.44 → 15.41 dB，SSIM 0.7333 → 0.7567，线上场景 shoe-37 就是这个版本）。两版留出视角不同、不能逐张对比，能对比的只有平均指标——这句话我写进了页面里。
 - 查出一个真实的评估缺陷并修掉：原先「留出 8 个视角」取的是最后 8 个，实测它们是一整段连续弧（约占环绕 70°）→ 这一段完全没有参与训练，而网页查看器是让访客自由环绕的，第一眼就撞上盲区。改成按相机绕主体的方位角均匀留出后，留出的 8 个视角散开到整圈。

- **数据一致性可到账**：数据库里的高斯点数 $\times 32$ 字节 = 资产实际字节数，逐字节吻合；上线后用 md5 比对线上与本地资产，两处入口一致（今天这次可访问性修复也是这么验的）。
- **像素级判空**：用「亮像素占比 + 标准差 + 重心」区分“真渲染出来”和“加载成功但全黑”。
- **后端是真的**：空库返回 200 而非 404、重复 slug 409、未知场景 404、限流 429 + Retry-After、带链接的留言 400 拒收（各种写法都覆盖）、UTF-8 往返、时区分桶、CORS 白名单——都有对应测试；XSS 载荷实测只以文本呈现、DOM 无注入节点。

已知边界（不想含糊过去）

- **自训场景的画质瓶颈在拍摄，不在算力（这是我这轮最想讲清楚的结论）**：同一批 43 张手机照片（COLMAP 注册 39 张，其中 6 张被清晰度检查判为糊片，实际参与训练 31 个视角），在云端 31 GB 显存上跑满 30000 步、全分辨率渲染，留出视角 PSNR 仍只有 14.4–15.4。**网页里能认出是鞋，但边缘偏软、背景有杂点**；同一个查看器加载官方示例是清晰的，说明问题不在查看器或部署。真正的解法是重拍（视角加密到 50–80 张、避免糊片、贴近平移、背景干净），而不是加高斯、加步数或换更大的卡——这三条我都实测否掉了。
- **本机显存上限（实测，它决定了能做什么）**：WSL 下 GPU 单进程可分配显存只有约 **1.5 GB**（标称 8 GB；驱动报的“空闲”数不可信），且与 Windows 可用内存强相关（实测 700 MB $\leftrightarrow$ 2.7 GB）。结论：**单物体 / 桌面级可行**（3–5 万高斯即可），**房间级不行**（参考实现需 100–300 万高斯）。已用公开数据集（Mip-NeRF 360，240 张真实照片、自带 COLMAP 位姿）把同一条管线跑通验证：流程全绿、跳过 SfM 只要 15 分钟，但只塞得下 6.6 万高斯，画质糊（PSNR 11.6），因此**主动没有上线**——证据与数据都在仓库里。
- **实拍场景一·运动鞋（43 张环拍）**：43 张手机环拍（COLMAP 注册 39 张，90.7%） $\rightarrow$ 降采样与亮度归一 $\rightarrow$ COLMAP $\rightarrow$ 云端 30000 步训练 $\rightarrow$.splat 上网。三个版本同屏呈现（原版 / 后处理版 / 剔糊片重训版：留出视角 PSNR 14.44 $\rightarrow$ 15.41 dB），并在简介里写清每版的差别；另有一个「保温杯」场景作为受限对照（43 张照片只有 15 个视角注册成功，PSNR 10.4），用来佐证“输入决定上限”。
- **实拍场景二·手柄包装盒（60 张环拍，目前指标最好的一批）**：60 张手机实拍（主摄 1x、锁 AE/AF、1920 $\times$ 1440） $\rightarrow$ COLMAP 注册 **60/60（100%）**、平均重投影误差 0.70 px $\rightarrow$ 云端 30000 步、全分辨率渲染 $\rightarrow$ 导出后按空间范围裁掉背景 $\rightarrow$ 线上资产 **38,160** 高斯；留出视角 **PSNR 17.23 dB / SSIM 0.8506**（训练 581 s、显存峰值 785 MB）。它说明同一套管线在「视角加密 + 拍摄规范」下能明显上一个台阶；紧贴主体的放射状亮纹来自视角数量与拍摄距离，后处理治不掉，只能靠继续加密视角。
- **免费额度的边界**：后端跑在 EdgeOne 云函数 + Neon 免费版 Postgres（计算空闲自动挂起、下次连接自动唤醒），月额度对个人作品站足够，但不是为高并发准备的。

下一步

- 重拍一轮（50–80 张、锁定曝光、避免糊片、背景干净） $\rightarrow$ 用同一套管线再训一次，把线上实拍场景的画质提上去。
- 按实验室安排打 C++ / Linux 基础：WSL2 环境已就绪（C++ 首个多文件工程与 CMake 项目已跑通，正在按课程推进）。
- 把方法对比页扩到更多家族（如 2DGS / SuGaR / 4DGS 的取舍），继续只写有实测支撑的结论。

仓库内含全部过程记录：[README.md](#)（技术栈与验收标准）、[PROGRESS.md](#)（模块进度与带命令输出的证据）、[pipeline/README.md](#)（照片 $\rightarrow$ COLMAP $\rightarrow$ 训练 $\rightarrow$ 导出上线的完整管线）。「云端 30000 步」的三组对照数据、留出视角缺陷的定位过程、云端函数的七条硬约束与部署踩坑都写在里面，可随时按文档复现。

本页数字均取自线上接口与本地实测（2026-10-09 复核）。